香港浸會大學
HONG KONG BAPTIST UNIVERSITY

COMP4127/COMP7850

Information Security

System Security

Dr. Kevin Wang 2023/2024

香港浸會大學
HONG KONG BAPTIST UNIVERSITY

COMP4127/COMP7850

Information Security

System Security

Dr. Kevin Wang 2023/2024

香港浸會大學
HONG KONG BAPTIST UNIVERSITY

COMP4127/COMP7850

Information Security

System Security

Dr. Kevin Wang 2023/2024

Learning outcomes

After this lecture you should be able to:

注射

1. describe buffer overflow attacks and SQL injection.
2. operate a SQL injection example.
3. name some network attacks e.g. ping-to-death/ SYN flooding and describe the principle behinds.
4. name some mechanism to defense against these attack.

Outline

- Host Security
  - Buffer Overflow
  - SQL Injection
- Network Security
  - Ping-to-death
  - SYN Flooding
  - DDoS
  - DNS Attack

Overview

Computer security goals include **confidentiality, data integrity, service availability, authenticity**, etc. The goal of an attacker is to break these goals.

所

Hacking, a coarse term, including but not limited to take the full control of a computer! 粗糙

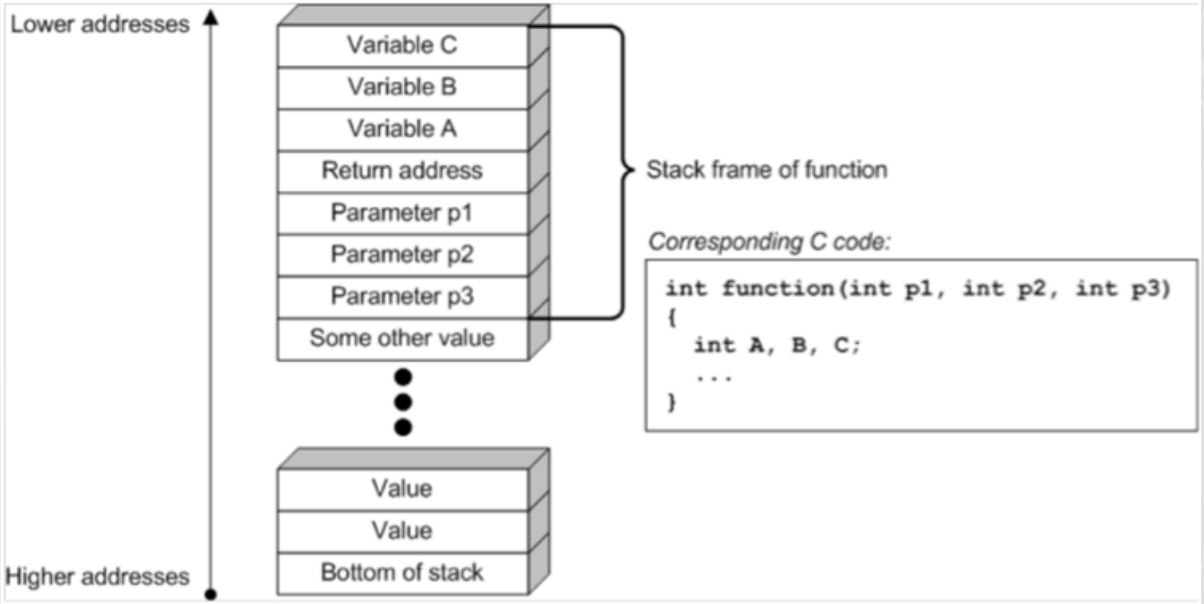
- A similar term: Intrusion 入侵

Common attacks via:

- Phishing 网络诈骗
- Malicious code 恶意的
- System vulnerability

Buffer Overflow Attacks

- When a program is executed, both code and data will be copied to memory. Code is placed at code segment, data is placed at data segment. Non-overlapped.
- When a function is called, a new memory will be reserved.

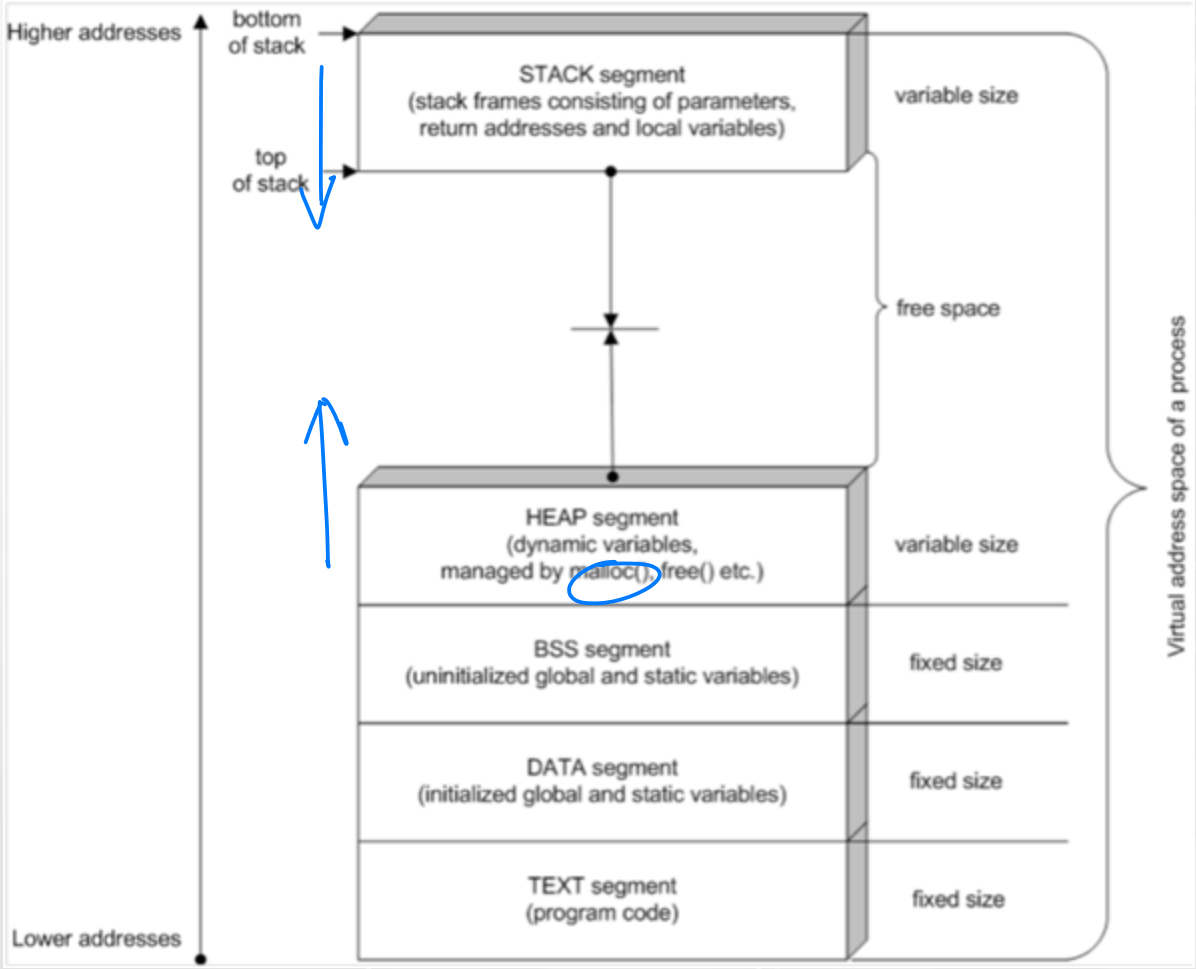


ref:https://www.drdobbs.com/security/anatomy-of-a-stack-smashing-attack-and-h/240001832?pgno=2

Buffer Overflow Attacks

There are two important storage: stack and heap.

- **Heap** is used to stored dynamic data (created by new/malloc), very often to be used in tree, linked list.
- **Stack** is used to stored local variables, static variables and return address.

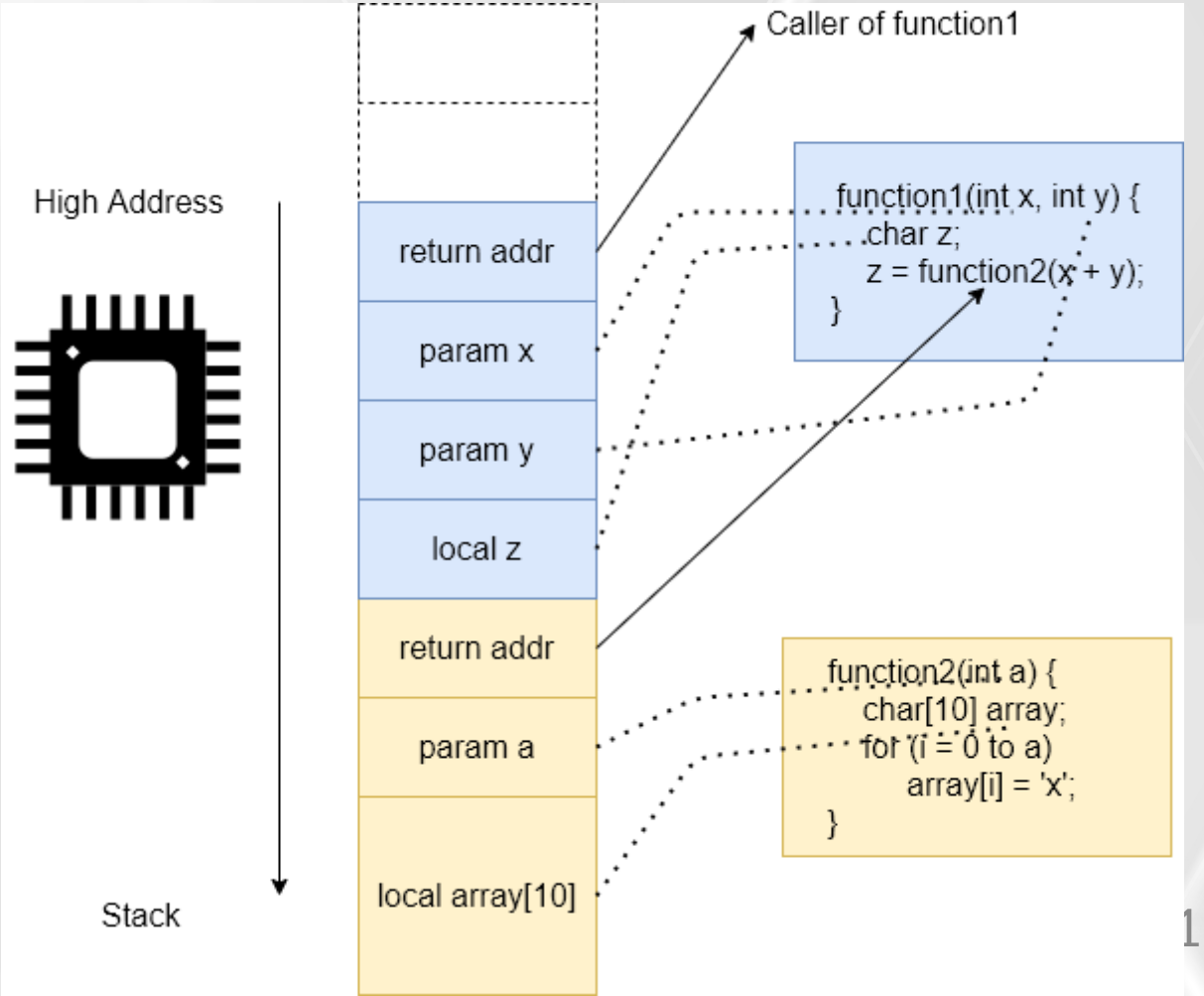


Buffer Overflow Attacks

- Return address stores the location of where the function should returns when the function is finished

In this example `function1` is the caller of `function2`. In `function2`'s memory space, it stores the address of the instruction `z=function2(x+y)`. When `function2` finishes, it returns to there.

💡 Imagine what will happen if `x+y>=10` ?



Buffer Overflow Attacks

A C-string is an array of char terminated by the special character `NULL`.

```
char name[50] = "Kevin Wang";
```

i	0	1	2	3	4	5	6	7	8	9	10	11 and onwards
name[i]	K	e	v	i	n		W	a	n	g	NULL	Does not matter

- A `NULL` character indicates the end of a string.

Consider C/C++ APIs like `gets` and `strcpy`

```
char buffer[30];
char dst[30];
gets(buffer);    //enter a string and put it in buffer
strcpy(dst, buffer); //copy the content from buffer to dst
```

type & store in the buffer.

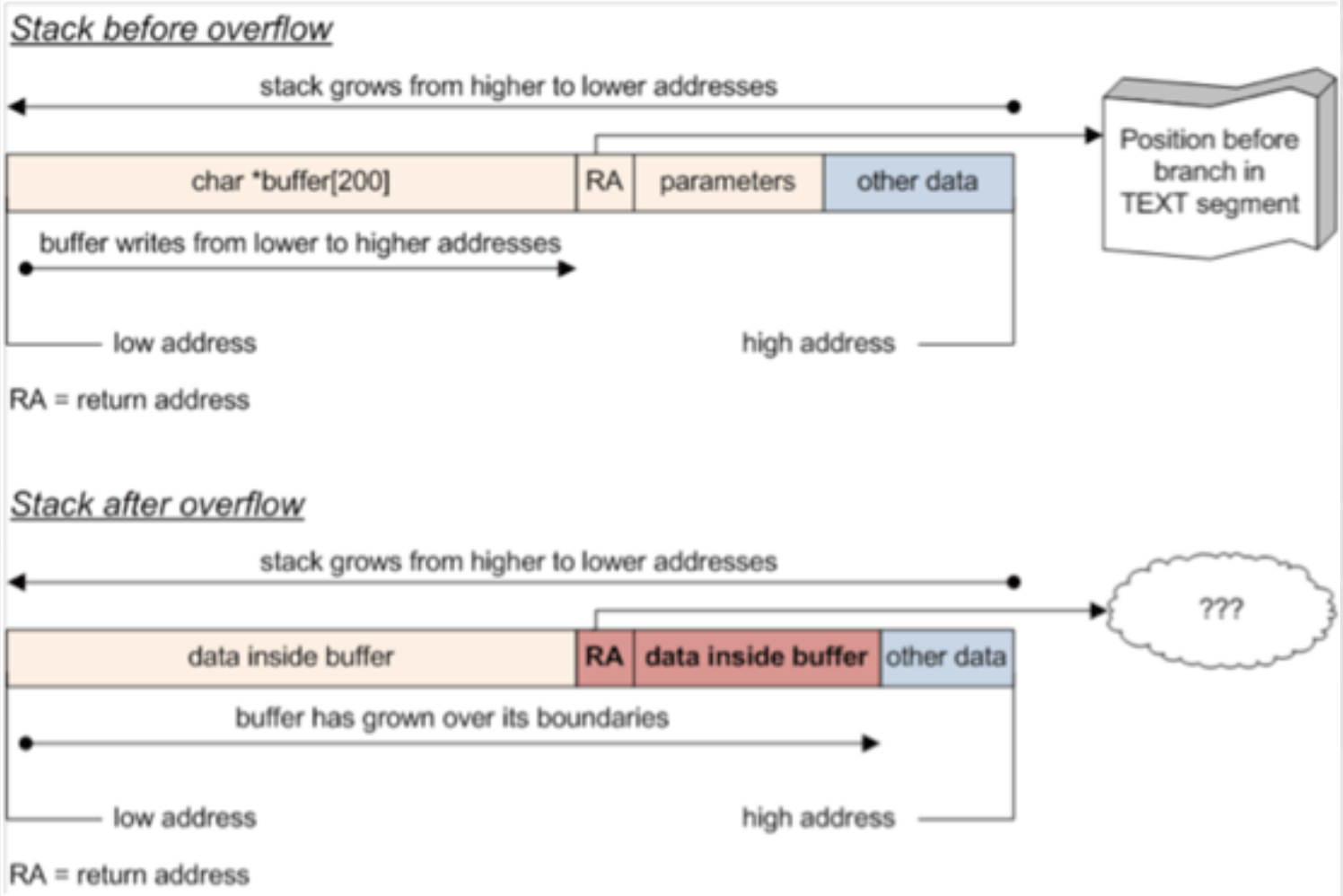
copy until "NULL" sand

Buffer Overflow Attacks

- Problems with `gets` and `strcpy`: not checking the length of the source and buffer.
- If the source text is too long, it will overflow the buffer and write to other variables, possibly including the return address.

💡 Tricks: to overflow the buffer with malicious code (shell code), overwrite the return address to the malicious code. The function will return to the malicious code instead of the function caller.

Buffer Overflow Attacks



Buffer Overflow Attacks

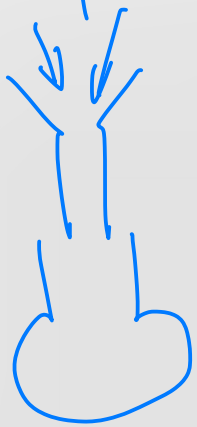
NOP: Doing nothing

Buffer Overflow Attacks

- The return address may not be precisely land on the starts of the shellcode. If it lands before the code, there must be some attacker's tricks to run the shellcode.
- **Sled**: a consecutive of `NOP` operations (or equivalent) placed before the shellcode. Attacker places a long sled into the buffer. If the return address returns to the exploited buffer, the series of `NOP` operations will *slide* the instruction pointer to the shell code.

```
0x3340 NOP
0x3341 NOP
0x3342 NOP
...
0x3367 NOP
# START OF EVIL CODE
0x3368 PLANT BOMB
0x3369 SMASH
0x336A IGNITE
```

Guide the pointer to evil code.



Buffer Overflow Attacks - Countermeasure

1. Avoid using high risk API.

Some APIs have a higher risk if used improperly. For example,

```
char buff[10];
strcpy(buff, source); //buffer overflow may occur if source is too long.
```

We can use some safer APIs.

```
strncpy(buff, source, 10); //max copy 10 characters
```

Risk Function	Safer Alternative
strcpy	strncpy
strcat	strncat
gets	fgets

Buffer Overflow Attacks - Countermeasure

2. Measure the length of the string first.

Before copying the string, measure the length of the source to avoid overflow.

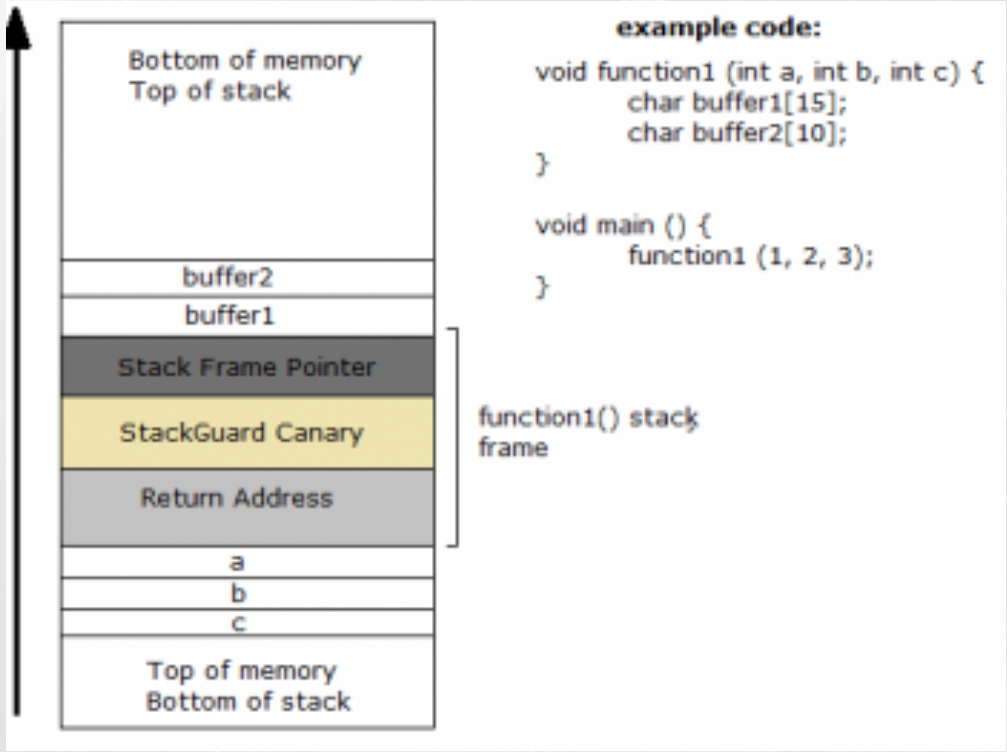
```
char buff[10];
int length = strlen(source);
if (length > 10)
  cout << "Too long!"
else
  strcpy(buff, source);
```

Buffer Overflow Attacks - Countermeasure

3. Compiler Assistant

The GCC compiler provide a set of options to use canaries.

1. Detection of sled
2. CPU assistant (XN)



SQL Injection

An authentication system may check a user password from a database, like

```
uName = getRequestString("username");
uPass = getRequestString("userpassword");

sql = 'SELECT * FROM Users WHERE Name ="' + uName + '" AND Pass ="' + uPass + '"'
```

If the statement returns 0 means the username and password does not match. If it returns non zero the user will be authenticate.

SQL Injection

What if the attacker types

- username: `" or ""="`
- password: `" or ""="`

The resulting SQL statement will become

```
SELECT * FROM Users WHERE Name ="" or ""="" AND Pass ="" or ""=""
```

This will always return true.

F || T ✓

F || T ✓

SQL Injection

利用 一批处理

Another way is to exploit the batch execution.

sql -- : comment

```
SELECT * FROM Users; DROP TABLE Suppliers
```

This will drop the table after selecting from users.

```
txtUserId = getRequestString("UserId");
txtSQL = "SELECT * FROM Users WHERE UserId = " + txtUserId;
```

Attacker set userID as `105; DROP TABLE Suppliers`

```
SELECT * FROM Users WHERE UserId = 105; DROP TABLE Suppliers;
```

SQL Injection - counter measure

- Write better program
- Data validation in front-end (e.g. check password does not contain " before calling the SQL)
- Use prepare statement in query instead.

```
$stmt = $dbh->prepare("INSERT INTO Customers (CustomerName,Address,City)
VALUES (:nam, :add, :cit)");
$stmt->bindParam(':nam', $txtNam);
$stmt->bindParam(':add', $txtAdd);
$stmt->bindParam(':cit', $txtCit);
$stmt->execute();
```

(example of a php code)

https://www.w3schools.com/sql/sql_injection.asp

Ping

Ping is a protocol to check the availability of an IP address over the ICMP protocol. The responder will reply a *Pong* after someone *ping* him. Usually one second one ping.

```
Pinging www.comp.hkbu.edu.hk [158.182.7.18] with 32 bytes of data:
Reply from 158.182.7.18: bytes=32 time<1ms TTL=64
Reply from 158.182.7.18: bytes=32 time<1ms TTL=64
Reply from 158.182.7.18: bytes=32 time<1ms TTL=64
Reply from 158.182.7.18: bytes=32 time<1ms TTL=64

Ping statistics for 158.182.7.18:
    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 0ms, Maximum = 0ms, Average = 0ms
```

Ping-of-death

Attackers ping a machine with a very fast rate, say 1000 times at a second. The server will be too busy.
Consider giving prank calls to a restaurant during lunch hour.

恶作剧

```
ping -i 0.04 www.comp.hkbu.edu.hk
PING www.comp.hkbu.edu.hk (158.182.7.18) 56(84) bytes of data.
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=1 ttl=37 time=1.18 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=2 ttl=37 time=1.69 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=3 ttl=37 time=1.88 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=4 ttl=37 time=1.94 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=5 ttl=37 time=2.05 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=6 ttl=37 time=1.87 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=7 ttl=37 time=1.83 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=8 ttl=37 time=1.88 ms
64 bytes from 158.182.7.18 (158.182.7.18): icmp_seq=9 ttl=37 time=1.70 ms
```

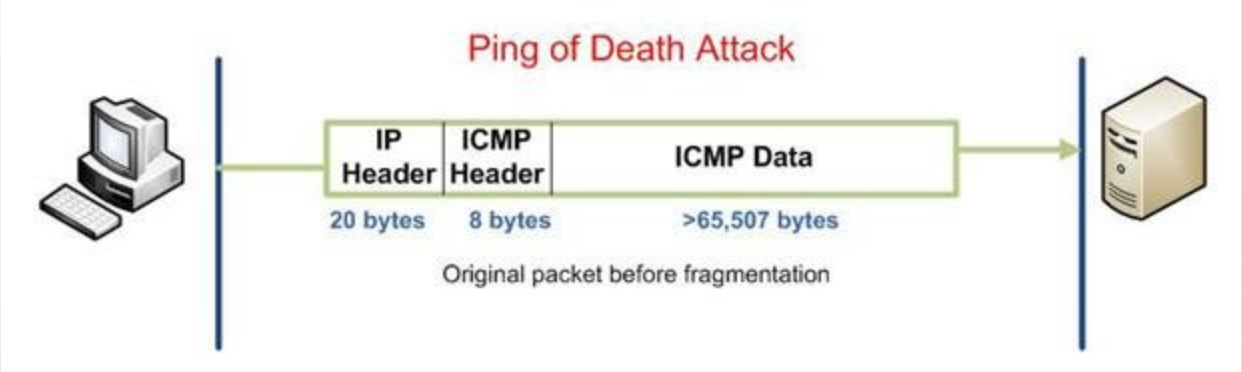
The argument `-i` specifies the interval of the ping (0.04s).

Ping-of-death v2

The basic version of ping-of-death is not an efficient attacks:

- Consume a lot of resources from the attacker.
- Easy to detect, easy to block.

A newer version of this attack is to override the ICMP packet size to more than tha max size allowed. A typical ICMP packet is 64 bytes only.



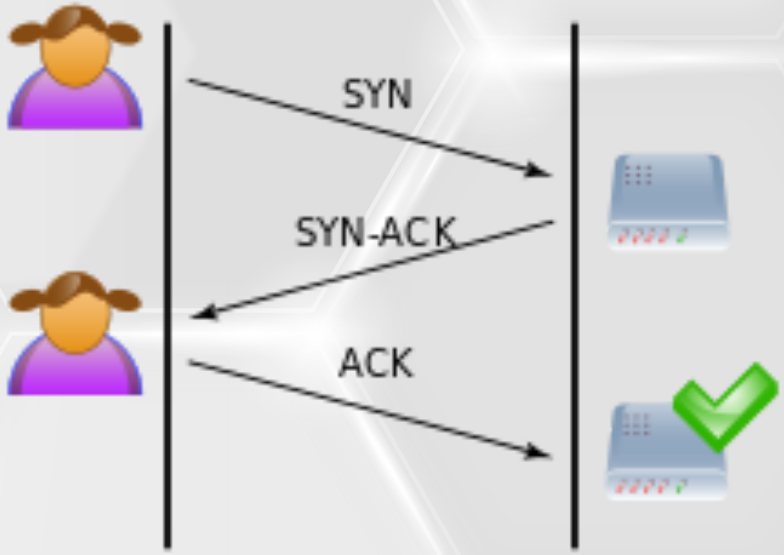
Under ICMP protocol we don't expect a packet larger than 65535 (max size of IPv4), the server might encounter a buffer-overflow.

A simple defense is to disable the ICMP protocol.

img https://www.pluralsight.com/blog/it-ops/ping-of-death-and-dos-attacks

TCP SYN

- Similar to ping, in TCP three-ways handshaking requires server to respond.
- To ensure that a TCP connection is stable, each connection begins with a three-way handshaking protocol. Once a SYN command is received at the server side, the server will response a SYN-ACK to the client. The client will respond ACK to the server to indicate that the client is reachable through the backward circuit.

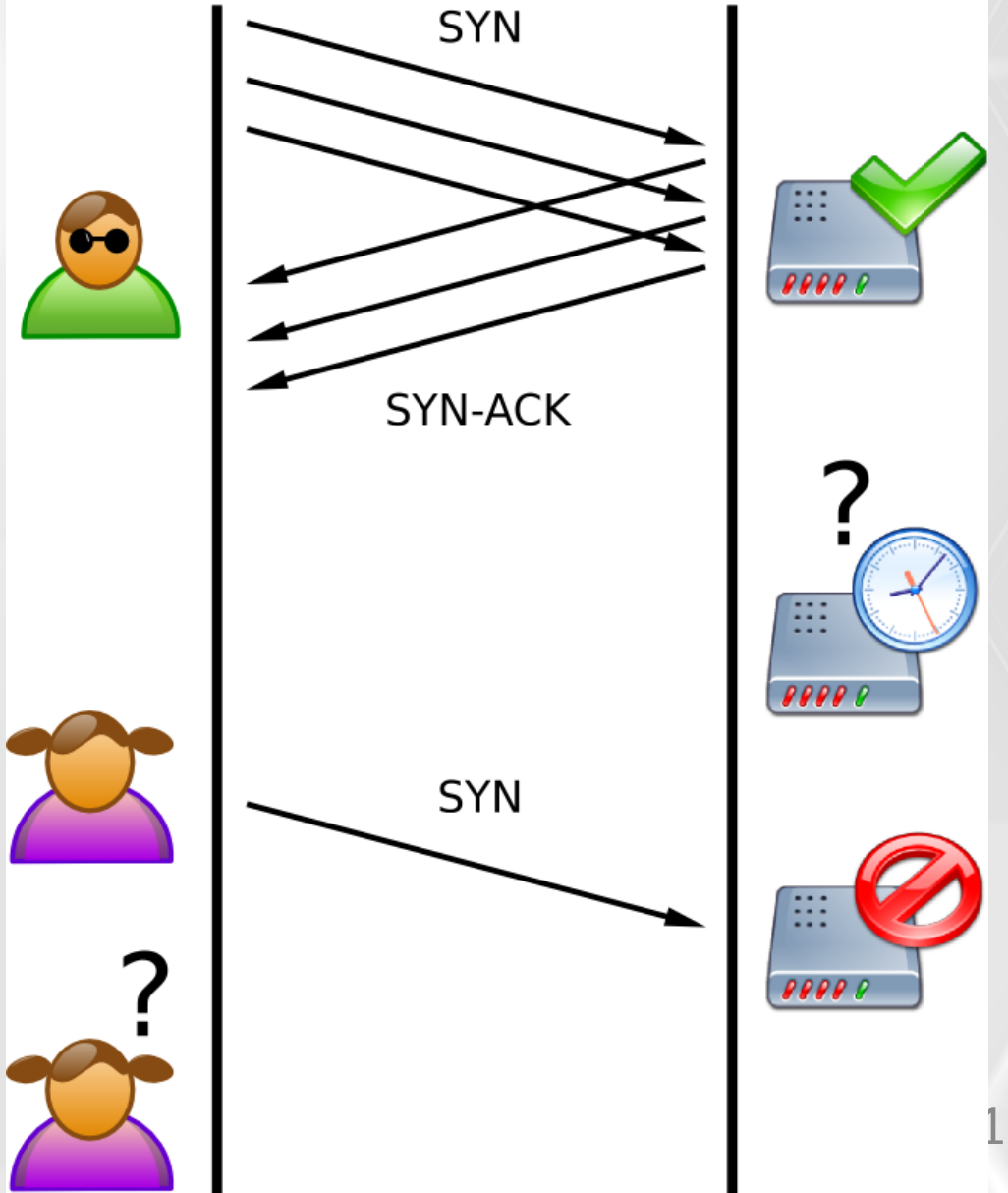


- If the backward circuit cannot reach the client, the server will know it after a TIMEOUT period.

img: wikipedia

SYN Flooding

- An attacker may issue a lot of SYN request to the server. Like ping server will reply SYN-ACK. But more than that, the server will open its connection and wait for the ACK.
- 黑名单 In both Ping and SYN the source can be blacklisted. But at the same time the attacker can apply **IP spoofing**.
- Unlike ping, SYN cannot be disabled for most server application (unless you are not running TCP!).



img: wikipedia

IP Spoofing

take IP address

- The IP packet header stores the routing information of a packet.
- IP Spoofing refers to falsify 歪曲 the packet header, e.g. to write a random source IP address.
- Server cannot block SYN packet directly.
- IPSec can be used to assert 断言 the correctness of IP header (unmodified in the midway). But it cannot be used to prevent SYN Flooding.

IPv4 Network Packet Headers

Version	IHL	Type of Service	Total Length	
Identification			Flags	Fragment Offset
Time To Live		Protocol	Header Checksum	
Source IP Address				
Destination IP Address				
Options				
Data				

lower the traffic on that port

img: https://www.imperva.com/learn/ddos/ip-spoofing/

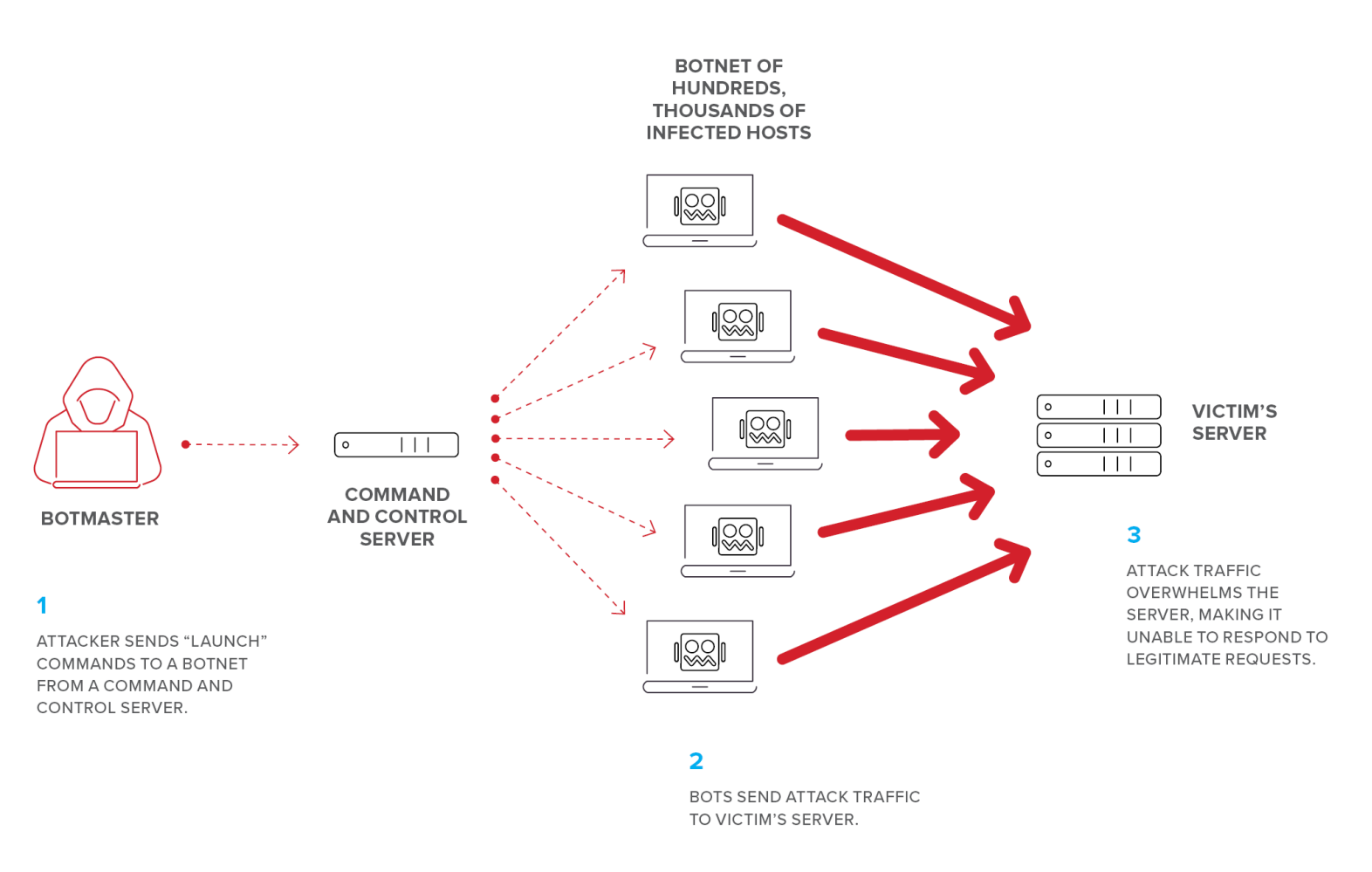
Amplification Attacks with IP Spoofing

效率 目的

- For efficiency sake, some protocols like *Network Time Protocol* (NTP), *Domain Name Service* (DNS), does not requires a TCP connection.
- Attackers can make requests for large volumes of replies from these services while putting the victims' IP addresses as the source addresses.
  - Imagine an evil person gives a prank call to a restaurant to order 10 people takeaway and ask them to deliver to a victim's home. Even worst, the attacker can order from many different restaurants!
- The attacker spends only a small amount of bandwidth while it is able to **amplify** the response by 10~100 times of traffic.

DDoS

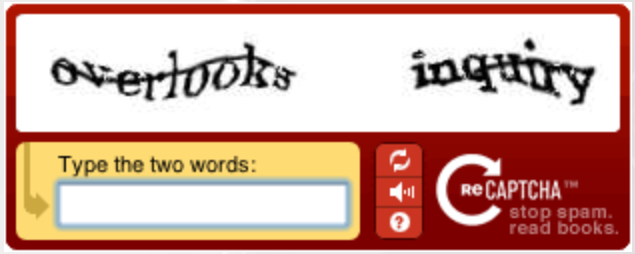
- Distributed Denial of Service (DDoS) is a upgrade version of a DoS attack. Attacker controls a lot of devices (through hacking perhaps) called **bots/zombies**. These bots forms a botnet.
- Since packets are sent from different routers, different direction, different countries, it is not easy block the packets.



DDoS

Counter Measures:

- Building Firewall/Intrusion Detection System (IDS) 闯入
  A firewall is a network equipment placed in the gateway to **block** malicious traffics which fulfill some predefined rules.
  An IDS is placed inside the network to **detect** the appearance malicious traffics and warn the network administrator.
- Adding challenges like Captcha
- Setting up Demilitarized Zone (DMZ) 非军事化
  Servers that has a higher risk/demand are placed in a DMZ. Other servers will be isolated from the DMZ.
- Purchasing other security service like cloudflare.



DNS Poisoning

- DNS server is responsible for translating a domain name to an IP address. There are a lot of DNS servers on earth to cache the IP address of different domains.
- A false entry on a DNS server will bring user to a bogus website. 造假

Domain Name	IP Address
abc.edu.hk	13.45.63.12
def.gov.hk	~~31.45.32.13~~ 35.22.22.44
evil.com	35.22.22.44

- Even worst DNS will share their entry to other DNS servers. This will spread the incorrect DNS entries.

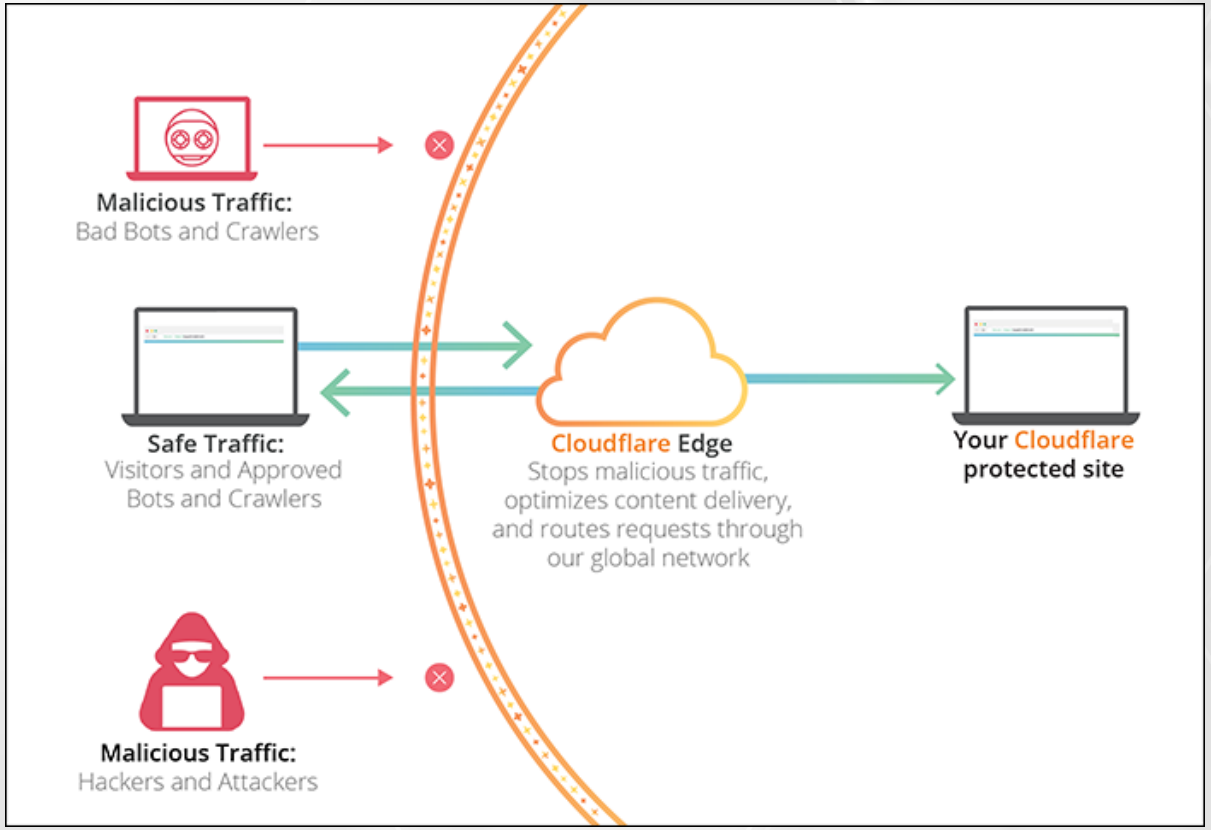
Counter measures:

1. Check certificate
2. Using multiple distributed DNS servers

Case study: Cloudflare

Cloudflare analyzes potential threats in visitor requests based on a number of characteristics:

- visitor's IP address,
- resources requested,
- request payload and frequency, and
- customer-defined firewall rules.



A DNS lookup or ping of a proxied Cloudflare subdomain returns Cloudflare IP addresses. Cloudflare masks your origin IP address for proxied (orange-clouded) DNS records so attackers cannot bypass Cloudflare and directly attack your origin web server.

Summary

- Host Security
  - Buffer Overflow
  - SQL Injection
- Network Security
  - Ping-to-death
  - SYN Flooding
  - DDoS
  - DNS Attack